

Submission Worksheet

Submission Data

Course: IT114-450-M2025

Assignment: IT114 - Milestone 3 - RPS

Student: Alex P. (ap2869)

Status: Submitted | **Worksheet Progress:** 100%

Potential Grade: 10.00/10.00 (100.00%)

Received Grade: 0.00/10.00 (0.00%)

Started: 7/31/2025 11:55:47 PM

Updated: 8/2/2025 8:23:24 PM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT114-450-M2025/it114-milestone-3-rps/grading/ap2869>

View Link: <https://learn.ethereallab.app/assignment/v3/IT114-450-M2025/it114-milestone-3-rps/view/ap2869>

Instructions

1. Refer to Milestone3 of [Rock Paper Scissors](#)
 1. Complete the features
2. Ensure all code snippets include your ucid, date, and a brief description of what the code does
3. Switch to the Milestone3 branch
 1. `git checkout Milestone3`
 2. `git pull origin Milestone3`
4. Fill out the below worksheet as you test/demo with 3+ clients in the same session
5. Once finished, click "Submit and Export"
6. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
 1. `git add .`
 2. ``git commit -m "adding PDF"`
 3. `git push origin Milestone3`
 4. On Github merge the pull request from Milestone3 to main
7. Upload the same PDF to Canvas
8. Sync Local
 1. `git checkout main`
 2. `git pull origin main`

Section #1: (1 pt.) Core Ui

Progress: 100%

≡ Task #1 (0.50 pts.) - Connection/Details Panels

Progress: 100%

📁 Part 1:

Progress: 100%

Details:

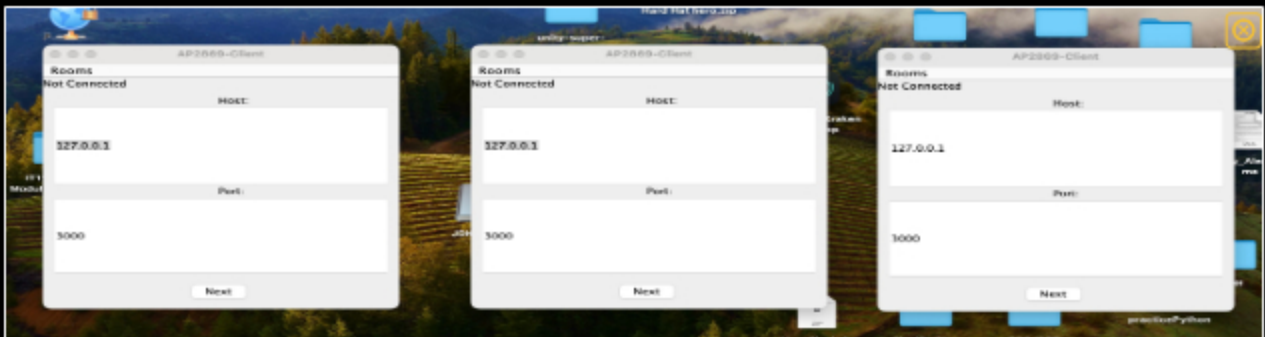
- Show the connection panel with valid data
- Show the user details panel with valid data

```

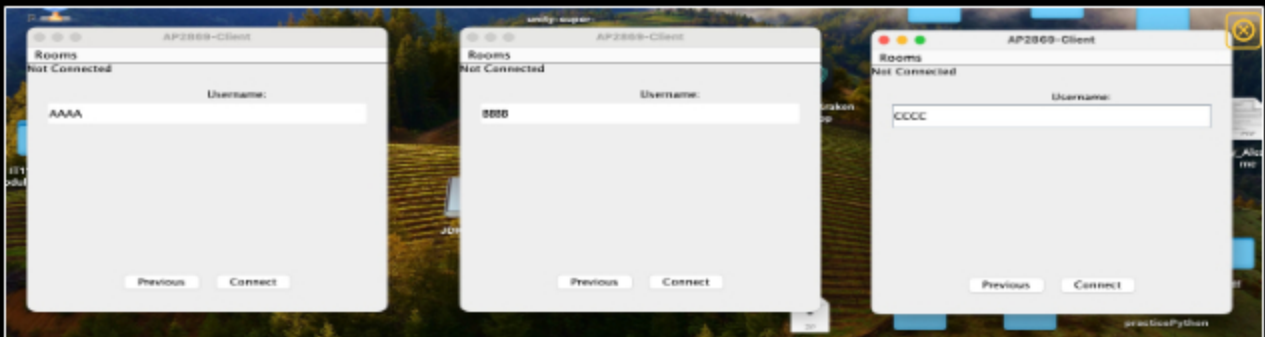
~/Desktop/NIIT/summer/114/IT-114-458
Milestone3-RockPaperScissors-project
$ ./build.sh Project

~/Desktop/NIIT/summer/114/IT-114-458
Milestone3-RockPaperScissors-project
$ ./run.sh Project server 3000
08/01/2025 16:24:16 [Project.Server.S
erver] (INFO):
  * Server Starting
08/01/2025 16:24:18 [Project.Server.S
erver] (INFO):
  * Server Listening on port 3000
08/01/2025 16:24:18 [Project.Server.R
oom] (INFO):
  > Room[lobby]: Created
08/01/2025 16:24:18 [Project.Server.S
erver] (INFO):
  * Server Created new Room lobby
08/01/2025 16:24:18 [Project.Server.S
erver] (INFO):
  * Server: Waiting for next client
  
```

server side of panel for project



host and port



Username of client

Saved: 8/1/2025 4:47:46 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code flow from recording/capturing these details and passing them through the connection process

Your Response:

the connection process begins with the client entering there host and port. but it is already

entered for them . then after next that when the client has to enter a username for them selves, the user name has to be longer then 3 characters. in this case the host is 127.0.0.1 and the server is 3000. after a username is entered thats when the client can connect to the server. tby connecting the client creates a socket connection to the server. thats when a inputstream and outputstream is established. thats when send(constant.connect send a connection request containing the data and username. the server thread then recives this and validates the username.

 Saved: 8/1/2025 4:47:46 PM

≡ Task #2 (0.50 pts.) - Ready Panel

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the button used to mark ready
- Show a few variations of indicators of clients being ready (3+ clients)



roomcreation by AAAA and BBBB,CCCC joining game room



all players ready

 Saved: 8/1/2025 5:06:39 PM

≡ Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for marking READY from the UI
- Briefly explain the code flow from receiving READY data and updating the UI

Your Response:

marking READY from the UI... the user clicks the ready button, client.sendreadystatus then sends ready status to the server, serverThread recives it, then setReady is clss and shown to all players that are in the room created. then the code auto checks if everyone is ready and auto starts the game when everyone is ready.

READY data and updating the UI... server sends the ready update by the players, onPlayerReady make the client recieve it. when a player is ready a box is shown on the bottom of the clients that are in the room. the game will not auto start if all the players arent ready.



Saved: 8/1/2025 5:06:39 PM

Section #2: (2 pts.) Project Ui

Progress: 100%

≡ Task #1 (0.67 pts.) - User List Panel

Progress: 100%

Details:

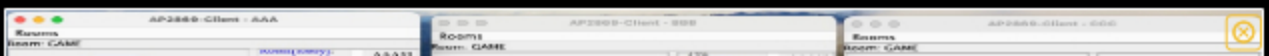
- Show the username and id of each user
- Show the current points of each user
- Users should appear in score order, sub-sort by name when ties occur
- Pending-to-pick users should be marked accordingly
- Eliminated users should be marked accordingly

Part 1:

Progress: 100%

Details:

- Show various examples of points (3+ clients visible)
 - Include code snippets showing the code flow for this from server-side to UI
- Show that the sorting is maintained across clients
 - Include code snippets showing the code that handles this
- Show various examples of the pending-to-pick indicators
 - Include code snippets showing the code flow for this from server-side to UI
- Show various examples of elimination indicators
 - Include code snippets showing the code flow for this from server-side to UI

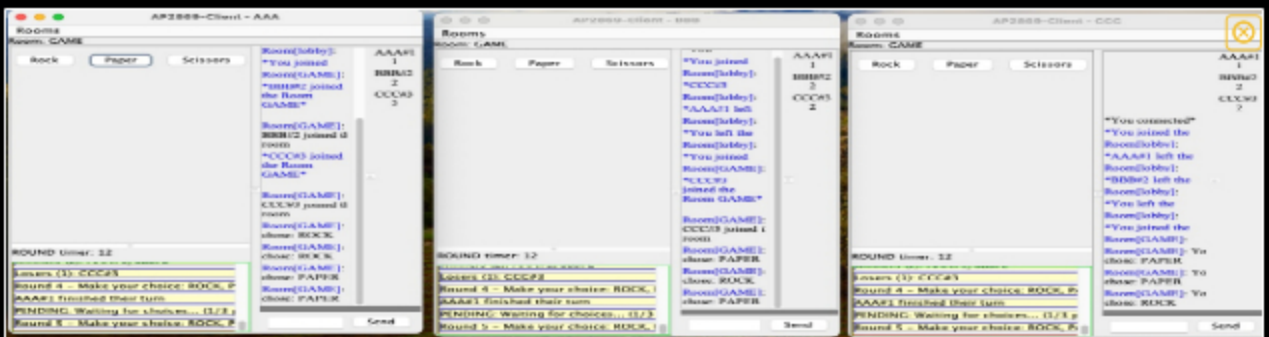




Points of players

```
String result;
if (choice1 == choice2) {
    result = String.format(format: "It's a tie! Both chose %s", choice1);
} else if (isWinningChoice(choice1, choice2)) {
    result = String.format(format: "%s wins with %s!", player1Name, choice1);
    player1.changePoints(points:1);
    sendPlayerPoints(player1);
} else {
    result = String.format(format: "%s wins with %s!", player2Name, choice2);
    player2.changePoints(points:1);
    sendPlayerPoints(player2);
}
<- #479-483 else
```

snippet of points maintained across clients and final



AAA has picked and the rest are pending to choose.

```
if (allPlayersChose()) {
    processRPSRound();
} else {
    You, 2 hours ago + updating Milestone 3 Project

    int playersReady = (int) clientsInRoom.values().stream().filter(ServerThread::isReady).count();
    int playersChosen = playerChoice.size();
    sendGameEvent(String.format(format: "PENDING: Waiting for choices... (%d/%d players have chosen)",
        playersChosen, playersReady));
}
<- #413-419 else
catch (Exception e) {
    LoggerUtil.INSTANCE.severe(message: "processRPSChoice exception", e);
}
```

snippet of pending choice



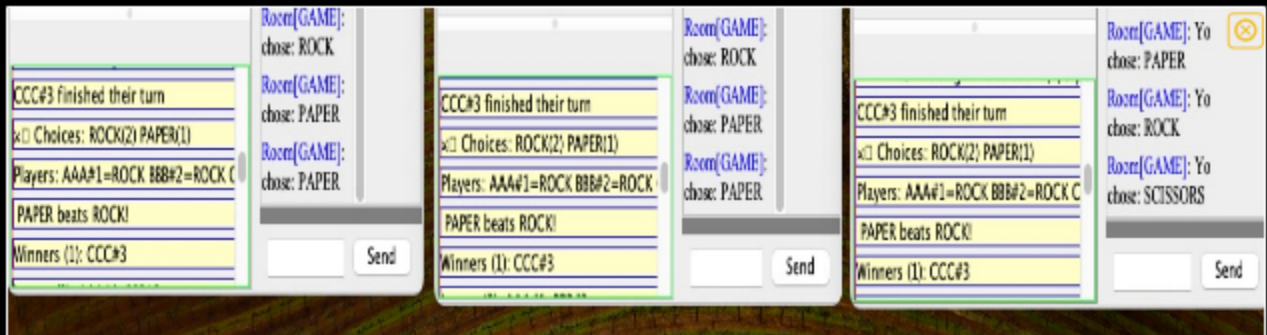
points maintained across clients and final

```
private void FinalScores() {
    StringBuilder scoreMessage = new StringBuilder(str:"** FINAL SCORES ** \n");

    List<ServerThread> sortedPlayers = clientsInRoom.values().stream()
        .sorted((p1, p2) -> Integer.compare(p2.getPoints(), p1.getPoints()))
        .collect(Collectors.toList());

    for (int i = 0; i < sortedPlayers.size(); i++) {
        ServerThread player = sortedPlayers.get(i);
        scoreMessage.append(String.format(format:"%s %s: %d points\n",
            player.getDisplayName(), player.getPoints()));
    } <- #295-299 for (int i = 0; i < sortedPlayers.size(); i++)
}
```

snippet of points maintained across clients and final



no elimination just CCC winning. the round

```
String winnerNames = winners.stream()
    .map(id -> clientsInRoom.get(id).getDisplayName())
    .collect(Collectors.joining(delimiter:", "));

String loserNames = losers.stream()
    .map(id -> clientsInRoom.get(id).getDisplayName())
    .collect(Collectors.joining(delimiter:", "));

sendGameEvent(String.format(format:"%s beats %s!", winningChoice, losingChoice));
sendGameEvent(String.format(format:"Winners (%d): %s", winners.size(), winnerNames));
sendGameEvent(String.format(format:"Losers (%d): %s", losers.size(), loserNames));
} <- #523-553 else
<- #486-554 private void processMultiPlayerRPS(List<Long> playerIds)
private void clearChoicesAndEndRound() {
    playerChoice.clear();
    onRoundEnd();
}
```

no elimination just winning. the round snippet



Saved: 8/1/2025 6:17:36 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for points updates from server-side to the UI
- Briefly explain the code flow for user list sorting
- Briefly explain the code flow for server-side to UI of pending-to-pick indicators
- Briefly explain the code flow for server-side to UI of elimination indicators

Your Response:

points updates from server-side to the UI.. when a player wins a round, player1.changeoint

updates the server side score, sendplayerpoints then will show the points to everyone in the gameroom. then onplayerpoints gives it to the clients and updates the scores on the board by their name. all scores are updated at the same time in real time.

user is sorting ... when the game ends FinalScore will call clientsInRoom.values().stream().sorted((p1, p2)Integer.compare(p2.getPoints(), p1.getPoints())).collect(Collectors.toList()); and will take the points and sort the players from least to greatest. then a message is sent to all players dtermining there score and ranking.

server-side to UI of pending-to-pick indicators.. when a player makes a choice player.settookturn will mark that the player made a choice. sendturnstatus will show the status to the players. while a player is waiting for their turn a pending message will show.

server-side to UI of elimination indicators.. no elimination for this game just winner and loser.

 Saved: 8/1/2025 6:17:36 PM

≡ Task #2 (0.67 pts.) - Game Events Panel

Progress: 100%

Details:

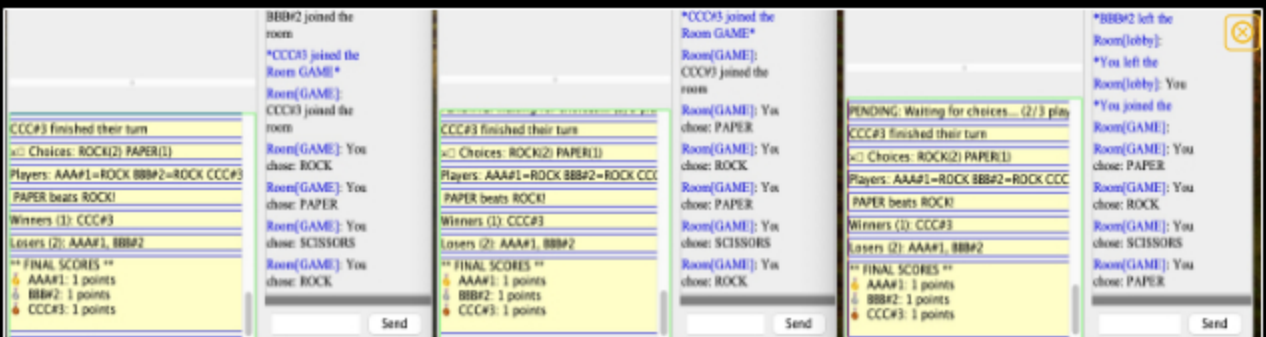
- Show the status of users picking choices
- Show the battle resolution messages from Milestone 2
 - Include messages about elimination
- Show the countdown timer for the round

Part 1:

Progress: 100%

Details:

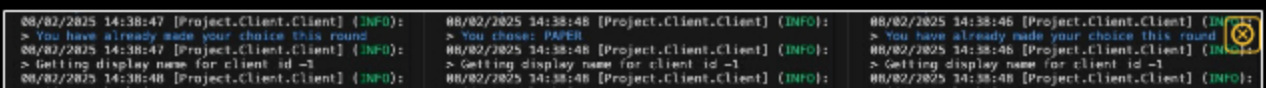
- Show various examples of each of the messages/visuals
- Show code snippets related to these messages from server-side to UI



The four screenshots show the following states:

- Left:** CCC#3 finished their turn. Choices: ROCK(2) PAPER(1). Players: AAA#1=ROCK BBB#2=ROCK CCC#3. PAPER beats ROCK! Winners (1): CCC#3. Losers (2): AAA#1, BBB#2. FINAL SCORES: AAA#1: 1 points, BBB#2: 1 points, CCC#3: 1 points.
- Second:** CCC#3 finished their turn. Choices: ROCK(2) PAPER(1). Players: AAA#1=ROCK BBB#2=ROCK CCC#3. PAPER beats ROCK! Winners (1): CCC#3. Losers (2): AAA#1, BBB#2. FINAL SCORES: AAA#1: 1 points, BBB#2: 1 points, CCC#3: 1 points.
- Third:** CCC#3 finished their turn. Choices: ROCK(2) PAPER(1). Players: AAA#1=ROCK BBB#2=ROCK CCC#3. PAPER beats ROCK! Winners (1): CCC#3. Losers (2): AAA#1, BBB#2. FINAL SCORES: AAA#1: 1 points, BBB#2: 1 points, CCC#3: 1 points.
- Right:** PENDING: Waiting for choices... (2/3) play. CCC#3 finished their turn. Choices: ROCK(2) PAPER(1). Players: AAA#1=ROCK BBB#2=ROCK CCC#3. PAPER beats ROCK! Winners (1): CCC#3. Losers (2): AAA#1, BBB#2. FINAL SCORES: AAA#1: 1 points, BBB#2: 1 points, CCC#3: 1 points.

users picking choices



The three screenshots show the following states:

- Left:** 08/02/2025 14:38:47 [Project.Client.Client] {INFO}: > You have already made your choice this round. 08/02/2025 14:38:47 [Project.Client.Client] {INFO}: > Getting display name for client id -1. 08/02/2025 14:38:48 [Project.Client.Client] {INFO}:
- Middle:** 08/02/2025 14:38:48 [Project.Client.Client] {INFO}: > You chose: PAPER. 08/02/2025 14:38:48 [Project.Client.Client] {INFO}: > Getting display name for client id -1. 08/02/2025 14:38:48 [Project.Client.Client] {INFO}:
- Right:** 08/02/2025 14:38:46 [Project.Client.Client] {INFO}: > You have already made your choice this round. 08/02/2025 14:38:46 [Project.Client.Client] {INFO}: > Getting display name for client id -1. 08/02/2025 14:38:48 [Project.Client.Client] {INFO}:

server side of users picking RPS and ending in a tie

code snippet of choice handling.

code of choices being picked and shown to other players



Progress: 100%

- Briefly explain the code flow for generating these messages and getting them onto the UI

Your Response:

processRPSchoice generates the status of the player, then it determines the current progress by counting the players that are ready.

then sendgameevent send a "pending" message to all the players that are awaiting their turn. depending on the number of players the message will include the number of players that have chosen and the number of players that havent choosen yet. this happens all through socket connections. then onGameEvent updates the UI panel to show the messages to the player.



Saved: 8/2/2025 3:10:31 PM

≡ Task #3 (0.67 pts.) - Game Area

Progress: 100%

Details:

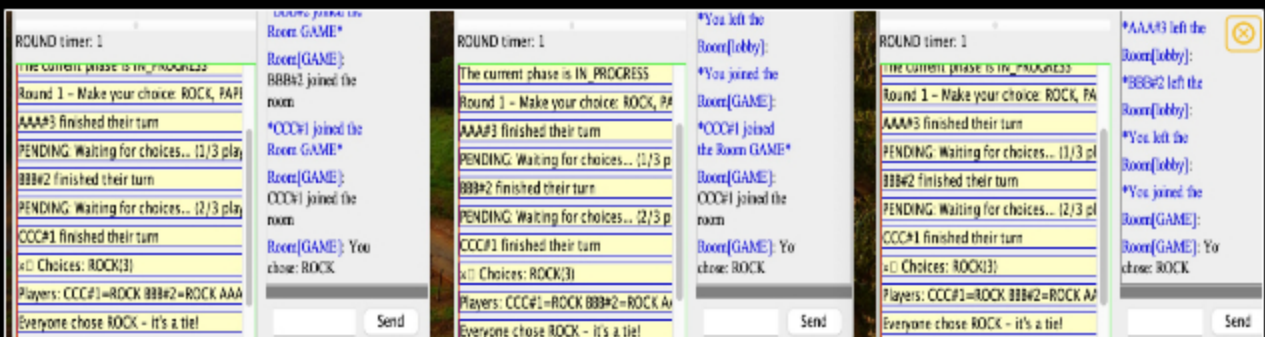
- UI should have components to allow the user to select their choice

📁 Part 1:

Progress: 100%

Details:

- Show various examples of selections across clients (3+ clients visible)
- Show the code related to sending choices upon selection
- Show the code related to showing visually what was selected



game in progress everyone chooses rock ending in a tie.

```
StringBuilder choiceAnnouncement = new StringBuilder(str:"x Choices: ");
for (Choice choice : Choice.values()) {
    if (choiceGroups.containsKey(choice)) {
        int count = choiceGroups.get(choice).size();
        choiceAnnouncement.append(String.format(format:"%s(%d) ", choice, count));
    }
}
// 465-470 for (Choice choice : Choice.values())
sendGameEvent(choiceAnnouncement.toString());
```

code sending choices upon selection

```
protected void handleTurnAction(ServerThread currentUser, String choice) {
    try {
        checkPlayerInRoom(currentUser);
        checkCurrentPhase(currentUser, Phase.IN_PROGRESS);
        checkIsReady(currentUser);

        if (currentUser.didTakeTurn()) {
            currentUser.sendMessage(Constants.DEFAULT_CLIENT_ID, message:"You have already made your choice this round");
            return;
        }

        // Process RPS choice
        processRPSChoice(currentUser, choice);

    } catch (NotReadyException e) {
        LoggerUtil.INSTANCE.severe(message:"handleTurnAction exception", e);
    }
}
```

cont code sending choices upon selection



user RPS options

```
private void processRPSChoice(ServerThread player, String textChoice) {
    try {
        Choice choice;
        try {
            if (textChoice.toUpperCase().equals(anObject:"SCISSOR")) {
                choice = Choice.SCISSORS;
            } else {
                choice = Choice.valueOf(textChoice.toUpperCase());
            }
        } catch (IllegalArgumentException e) {
            player.sendMessage(Constants.DEFAULT_CLIENT_ID, message:"Please choose ROCK, PAPER, or SCISSORS");
            return;
        }

        playerChoice.put(player.getClientId(), choice);
        player.sendMessage(Constants.DEFAULT_CLIENT_ID, "You chose: " + choice);
        player.setTookTurn(tookTurn:true);
        sendTurnStatus(player, tookTurn:true);
    }
}
```

snippet related to showing visually what was selected



Saved: 8/2/2025 4:08:00 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for selecting a choice and having it reach the server-side
- Briefly explain the code flow for receiving the selection for the current player to update the UI

Your Response:

when players choose either RPS as an option, the ui calls clients.senddoturn which chooses rock and send the choice made over the network then handleturnaction recives it makes sure the

and send the choice made over the network then handcraft a message that receives it, makes sure the player is valid and calls processRPSChoice which stores the players choice in playerChoice, afterwards setztookturn marks the player as chosen.

for a player that chooses 'rock' the server sends the message "you choose rock" message of confirmation to the player, then it is shown to all the players. ui then shows a green checkmark on the players who have chosen, which at the same time a player is choosing, the players who are not choosing are shown the message "pending" awaiting their turn.



Saved: 8/2/2025 4:08:00 PM

Section #3: (4 pts.) Project Extra Features

Progress: 100%

Task #1 (2 pts.) - Extra Choices

Progress: 100%

Details:

- Setting should be toggleable during Ready Check by session creator
 - (Option 1) Extra choices are available during the full session
 - (Option 2) Only activate extra options at different stages (i.e., last 3 players remaining)
- There should be at least 2 extra options for rps-5

Part 1:

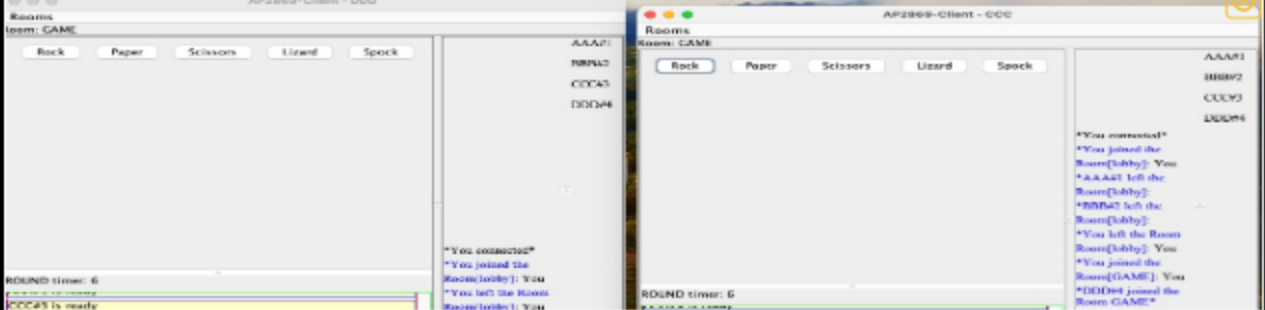
Progress: 100%

Details:

- Show the Ready Check screen with the option for the host (3+ clients must be visible)
 - Show the related code that makes this interactable only for the host
- Show the play screen with the extra options available
 - Show the related code for the UI and handling of these extra options (including battle logic)



players with more than 3 choices available during full session



players with more than 3 choices available during full session

```
String[] choices = {"Rock", "Paper", "Scissors", "Lizard", "Spock"};

for (String choice : choices) {
    JButton button = new JButton(choice);
    button.addActionListener(_ -> {
        try {
            Client.INSTANCE.sendDoTurn(choice.toLowerCase());
        } catch (IOException e) {
            e.printStackTrace();
        }
    }); // #22-28 button.addActionListener
    buttonPanel.add(button);
} // #29-30 for (String choice : choices)

this.add(buttonPanel); // You, 2 weeks ago • Adding milestone 3 Baseline files
} // #14-33 public PlayView(String name)
```

snippet of choices code

```
private boolean isWinningChoice(Choice choice1, Choice choice2) {
    return (choice1 == Choice.ROCK && choice2 == Choice.SCISSORS) ||
           (choice1 == Choice.ROCK && choice2 == Choice.LIZARD) ||
           (choice1 == Choice.PAPER && choice2 == Choice.ROCK) ||
           (choice1 == Choice.PAPER && choice2 == Choice.SPOCK) ||
           (choice1 == Choice.SCISSORS && choice2 == Choice.PAPER) ||
           (choice1 == Choice.SCISSORS && choice2 == Choice.LIZARD) ||
           (choice1 == Choice.LIZARD && choice2 == Choice.PAPER) ||
           (choice1 == Choice.LIZARD && choice2 == Choice.SPOCK) ||
           (choice1 == Choice.SPOCK && choice2 == Choice.SCISSORS) ||
           (choice1 == Choice.SPOCK && choice2 == Choice.ROCK);
} // #533-544 private boolean isWinningChoice(Choice choice1, Choice choice2)
```

code battle logic



Saved: 8/2/2025 5:35:20 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code for the host's option to toggle this feature
- Briefly explain the code related to handling these options including how it's handled during the battle logic
- Note which option you went with in terms of activating the choices

Your Response:

the host dosnt have options of the extra features because the extra options are avaiable for all players during the full session.

the battle modes handles through the same method, and that method is processRPSchoice. then

it converts any valid choice string into the enum value, and that's when winning choice applies the rules. the rules add 7nmore possibilities of winning while making a choice.

 Saved: 8/2/2025 5:35:20 PM

≡ Task #2 (2 pts.) - Choice cooldown

Progress: 100%

Details:

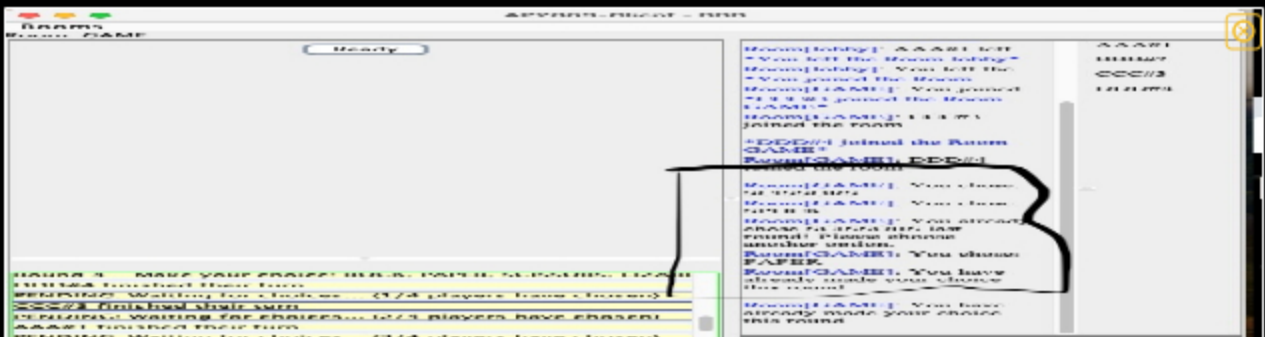
- Setting should be toggleable during Ready Check by session creator
- The choice on cooldown must be disable on the UI for the User

Part 1:

Progress: 100%

Details:

- Show the Ready Check screen with the option for the host (3+ clients must be visible)
 - Show the related code that makes this interactable only for the host
- Show a few examples of the play screen with the choice on cooldown
 - Show the related code for the UI and handling of the cooldown and server-side enforcing it



cool down where I couldnt choose scissor the third time because I already chose it the first

```
if (cooldownEnabled) {  
    Choice lastChoice = playerLastChoice.get(player.getClientId());  
    if (lastChoice != null && lastChoice == choice) {  
        player.sendMessage(Constants.DEFAULT_CLIENT_ID,  
            String.format(format:" You already chose %s last round! Please choose another option.", choice));  
        return; // Block the choice  
    } <- #444-448 if (lastChoice != null && lastChoice == choice)  
} <- #442-449 if (cooldownEnabled)
```

snippet of cool down and ui enforcement

```
private void processRPSChoice(ServerThread player, String textChoice) {  
    try {  
        Choice choice;
```

```

try {
    // Handle legacy and new choices
    String upperChoice = textChoice.toUpperCase();
    if (upperChoice.equals(anObject:"SCISSOR")) {
        choice = Choice.SCISSORS;
    } else {
        choice = Choice.valueOf(upperChoice);
    }
} catch (IllegalArgumentException e) {
    player.sendMessage(Constants.DEFAULT_CLIENT_ID, message:"Please choose ROCK, PAPER, SCISSORS, LIZARD, or SPOCK");
    return;
}

```

server side enforcing it



Saved: 8/2/2025 8:12:01 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code for the host's option to toggle this feature
- Briefly explain the code related to handling and enforcing the cooldown period (include how this is recorded per user and reset when applicable)

Your Response:

the host dosnt have an option to toggle this feature becasue the toggles are avaiable to all players throughout the 5 rounds of the game.

map<long is used to tack each players previous choice. when each round ends, the clearchoiceandendround method loops through all current choices and stores them as last choices. by doing this it creates a record that will be used for cool down in the next round.

the cooldown reset occurs when the user discoonects from the game, when a new game start and when a game ends . this prevents data from goin across different games and making sure that each new session starts fresh.



Saved: 8/2/2025 8:12:01 PM

Section #4: (2 pts.) Project General Requirements

Progress: 100%

Task #1 (1 pt.) - Away Status

Progress: 100%

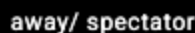
Details:

- Clients can mark themselves away and be skipped in turn flow but still part of the game
- The status should be visible to all participants
- A message should be relayed to the Game Events Panel (i.e., Bob is away or Bob is no

- The user list should have a visual representation (i.e., grayed out or similar)

Progress: 100%

- Show the UI button to toggle away
- Show the related code flow from UI to server-side back to UI for showing the status
- Show the related code flow for sending the message to Game Events Panel
- Show various examples across 3+ clients of away status (including Game Events Panel messages)
- Show the code that ignores an away user from turn/round logic



Saved: 8/2/2025 7:51:31 PM

Progress: 100%

- Briefly explain the code flow for the away action from UI to server-side and back to UI
- Briefly explain how the server-side ignores the user from turn/round logic

i didnt undertand this part of the assignment and had difficult applying it to my game . becasue i would assume that away would be the same as spectator.

Saved: 8/2/2025 7:51:31 PM

Progress: 100%

Details:

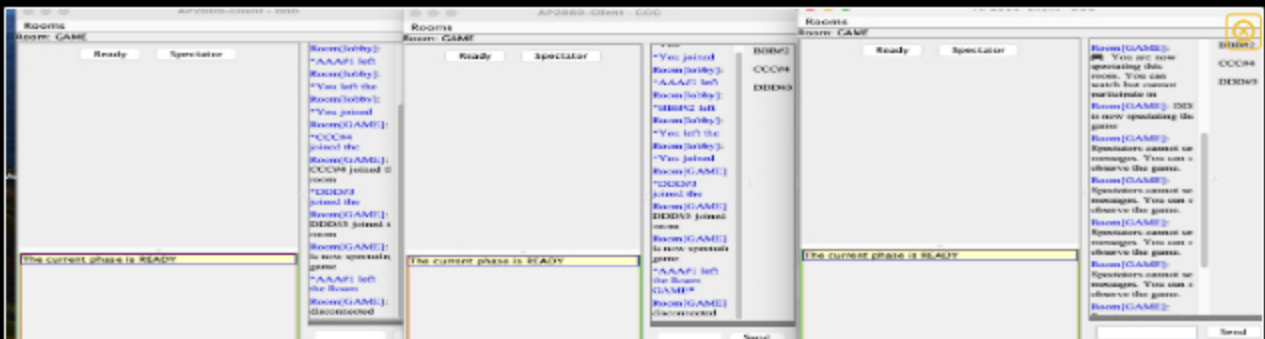
- Spectators are users who didn't mark themselves ready
 - Optionally you can include a toggle on the Ready Check page
- They can see all chat but are ignored from turn/round actions and can't send messages
- Spectators will have a visual representation in the user list to distinguish them from other players
- A message should be relayed to the Game Events Panel that a spectator joined (i.e., during an in-progress session)

Part 1:

Progress: 100%

Details:

- Show the UI indicator of a spectator (visual and message)
- Show the related code flow from UI to server-side back to UI for showing the status
- Show the related code flow for sending the message to Game Events Panel
- Show various examples across 3+ clients of spectator status (including Game Events Panel messages)
- Show the code that ignores a spectator from turn/round logic
- Show the code that prevents spectators from sending messages (server-side)
- Show the spectator's view of the session
- Show the code related to the spectator seeing the session data (including things participants won't see)



player DDD is a spectator and when trying to send message it didn't let it. others can see it either

```
protected void handleTurnAction(ServerThread currentUser, String choice) {  
    try {  
        if (currentUser.isSpectator()) {  
            currentUser.sendMessage(Constants.DEFAULT_CLIENT_ID,  
                message:"Spectators cannot make game choices. You can only observe the game.");  
            return;  
        }  
    }  
}
```

code that block specator from sending messages.

```
protected boolean sendMessage(long clientId, String message) {  
    Payload payload = new Payload();
```

```

payload.setPayloadType(PayloadType.MESSAGE);
payload.setMessage(message);
payload.setClientId(clientId);
return sendToClient(payload);
} <- #255-261 protected boolean sendMessage(long clientId, String message)

```

send message whether u are a spectator or player.

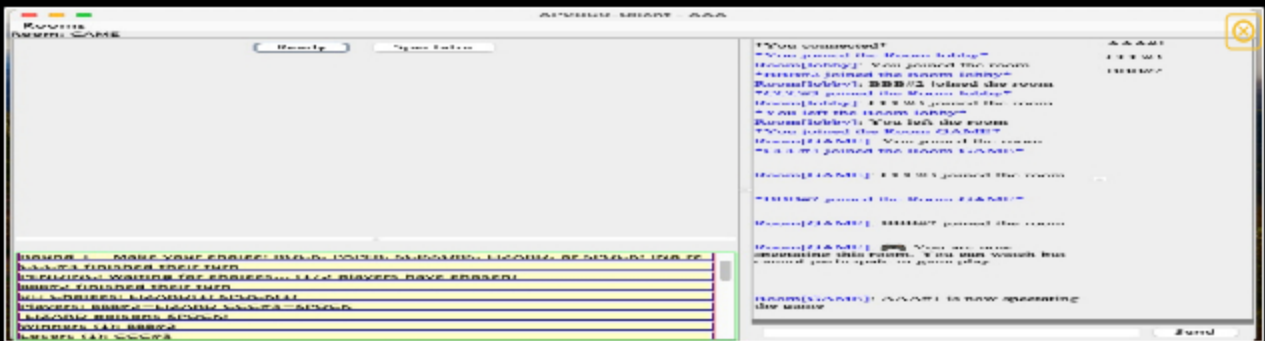
```

protected void handleSpectatorJoin(ServerThread client) {
    client.setSpectator(spectator:true);
    client.setReady(isReady:false);

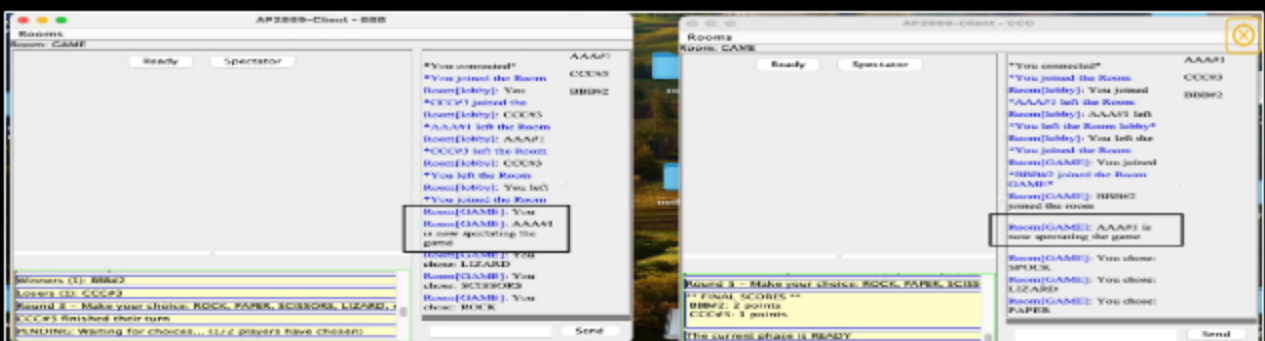
    client.sendMessage(Constants.DEFAULT_CLIENT_ID,
        message:" You are now spectating this game. You can watch but cannot participate in game.");
} <- #464-471 protected void handleSpectatorJoin(ServerThread client)

```

code related to the spectator seeing the session data (including things participants won't see)



spec view while game is in round 1 session



message sent to other players confieming player AAA is a spectator



Saved: 8/2/2025 8:23:24 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for the spectator logic from server-side and to UI
- Briefly explain how the server side ignores the user from turn/round logic

- Briefly explain how the server-side ignores the user from turn/round logic
- Briefly explain the logic that prevents spectators from sending a message
- Briefly explain the logic that shares extra details to the spectator (information normal participants won't see)

Your Response:

code flow for the spectator logic from server-side and to UI ... when a client clicks the spectator button, a slash command is sent to the server. the server thread then process this command in the processpayload method. after this is handled the clients isSpecator turns true and then boom they are a spectator. also a message for confirmation is sent to the client. a message is also sent to the rest of the players saying that you are a spectator

server-side ignores the user from turn/round logic... the server excludes spectators from gameplay through filtering. a filter was added to setturndorder excluding spectaor from turn rotation. allplayerschse count all the players that are not spectators.

logic that prevents spectators from sending a message... spectator message blocking happens in room by relay() method. and if statment is added to check if the player is a spectator or no before sending a message. if confirmed you are a spectator it block you from messaging. but messages from other players are visable for the spectator.

logic that shares extra details to the spectator (information normal participants won't see)... spectators will see a message for only them stating they are a spectator and that the game has started.



Saved: 8/2/2025 8:23:24 PM

Section #5: (1 pt.) Misc

Progress: 100%

≡ Task #1 (0.33 pts.) - Github Details

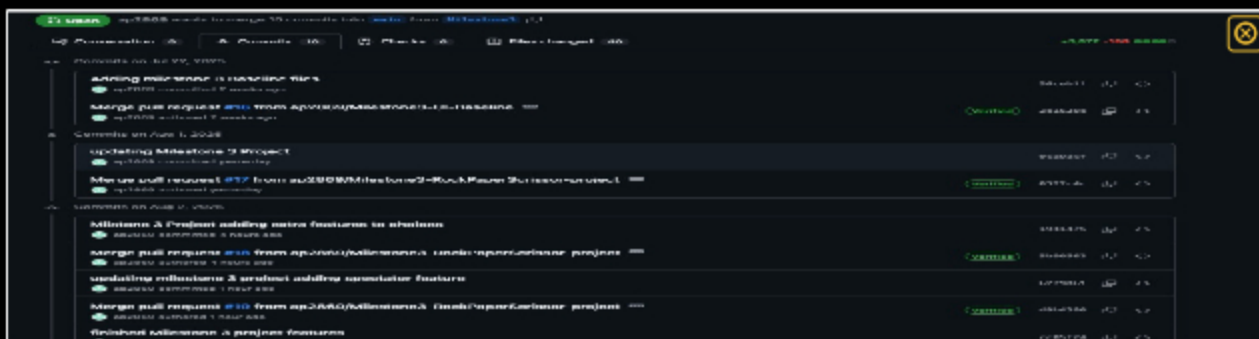
Progress: 100%

📁 Part 1:

Progress: 100%

Details:

From the Commits tab of the Pull Request screenshot the commit history



Milestone 3 commits



Saved: 8/2/2025 8:09:37 PM

Part 2:

Progress: 100%

Details:

Include the link to the Pull Request for Milestone3 to main (should end in `/pull/#`)

URL #1

<https://github.com/ap2869/IT-114-450/pull/21>



URL

<https://github.com/ap2869/IT-114>



Saved: 8/2/2025 8:09:37 PM

Task #2 (0.33 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click **Projects** and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary

Projects · IT-114-450

total 39 hrs 6 mins

7 hrs 23 mins over the Last 7 Days in IT-114-450 under all branches.

waka time

Files

4 hrs 28 mins	Project/Server/GameRoom.java
22 mins	Project/Server/ServerThread.java
17 mins	Project/Sandbox/PlayViewSandbox.java
16 mins	Project/Client/Views/PlayView.java
14 mins	Project/Server/GameRoom.class
13 mins	Project/Server/Room.java
10 mins	Project/Server/BaseGameRoom.java
8 mins	RPS/Sandbox/PlayViewSandbox.java
8 mins	Project/Client/Views/ReadyView.java

Branches

6 hrs 19 mins	Milestone3-
	RockPaperScissor-project
1 hr 3 mins	Milestone3

waka files



Saved: 8/2/2025 8:11:38 PM

≡ Task #3 (0.33 pts.) - Reflection

Progress: 100%

⇒ Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

by doing this project it taught me the fundamentals of client-server architecture, where clients/users handle UI interactions while the servers manage game state. the project also showed real examples of network programming concepts. things such as socket communication multiplayer interaction client and server side. since java is one of the languages that i want to advance in, it also has some software engineering practices.that could be really helpfull in the future and in the real world.



Saved: 8/2/2025 6:45:54 PM

⇒ Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

the easiest part of the assignment was basically everything. i was given basically 90 percent of the code that had to be done and i just had to impliment the game that i was doing. in this case it was rock paper scissor. it was pretty simple after i got the hand of it and had everything in place.



Saved: 8/1/2025 7:57:24 PM

Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

the hardest part of the assignment was coming up with the game and figuring out how the components would work. i had difficult implimentsing more then 3 players to a rock paper scissor game because usually its just 2 players so i had a difficult time trying to impliment more then 2 players but i got it done.



Saved: 8/1/2025 6:20:59 PM